MEMORANDUM

To: Charlie Refvem, Lecturer, Department of Mechanical Engineering, Cal Poly SLO
crefvem@calpoly.edu

From: Quinn Patterson
qpatters@calpoly.edu
Jake Rowen
jhrowen@calpoly.edu

Team: 23

Date: 3/05/26

RE: **Lab 0x06 Memo**

Table of Contents

1. Memo Body
2. Source Code

Memo Body

The state estimator will be useful tool to control Romi's speed when not in line following mode. We used the IMU mode to access gyroscope and accelerometer data. These measurements coupled with encoder counts and motor command voltages allow us to make useful state predictions of Romi. To get a strong baseline about where Romi should be in testing, we used our line follower code to have Romi trace a circle. Our predicted location was fairly accurate, only deviating at beginning of the data. This deviation is to be expected since the initial input matrix for the state estimator is all zeros, meaning the estimator needs data accurately estimate the state. Our track path can be viewed in the figure below. The observed state consists of Arc length, heading angle, and rotational velocities for each motor. The first two can be used to estimate the location of the robot, as we did in our graph.

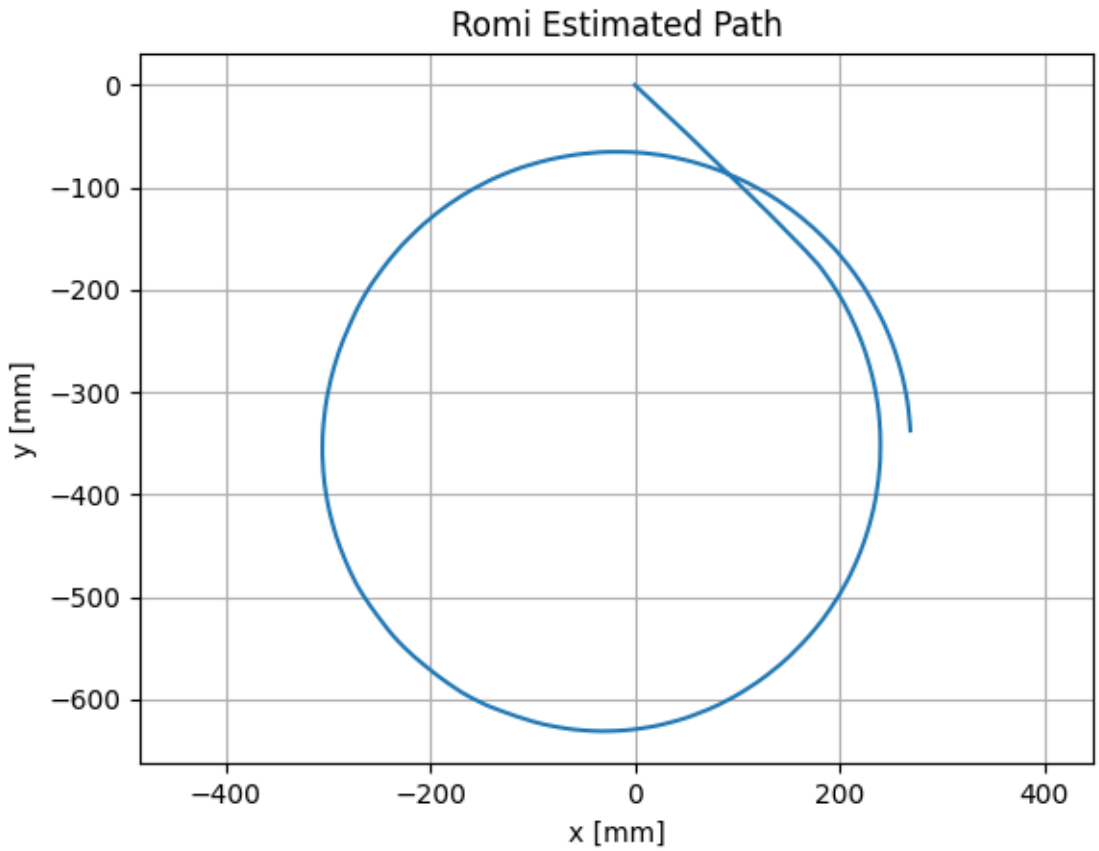

Figure 1. Romi State Estimation of Circular Path

We implemented two new tasks: a read sensor task and state estimator task. Read sensor pulls data from the IMU, Encoders, and Motors and stores them in shares for use in our u* matrix. They are then transferred via shares to the state estimator task which handles our estimation. An updated diagram of our tasks and FSMs can be viewed below. A class was written for the IMU that is references in the read sensor task. Additionally, two new queues are used for storing arc length and heading angle data from the state estimation task, which we used to create the above plot. The parameters from most recent estimated state are also stored in shares however are currently unused in our code.

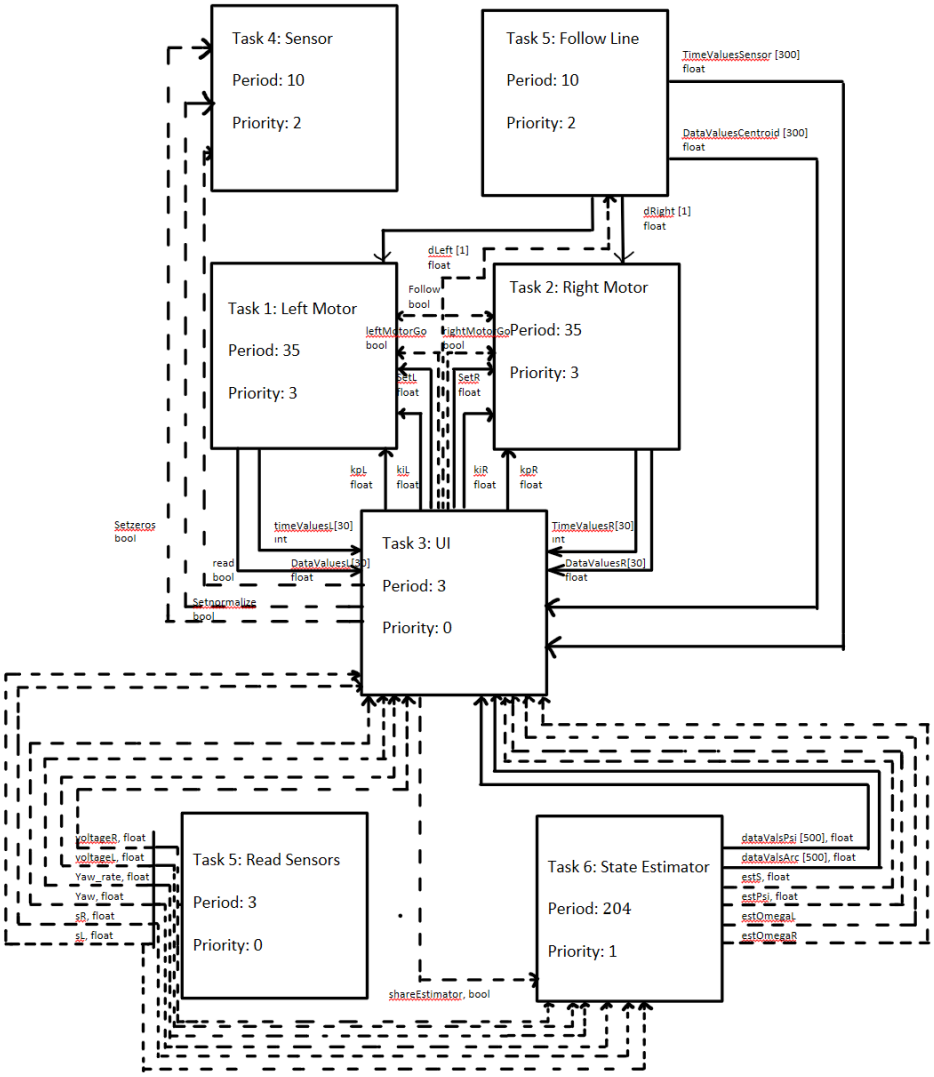

Figure 2. Task Diagram

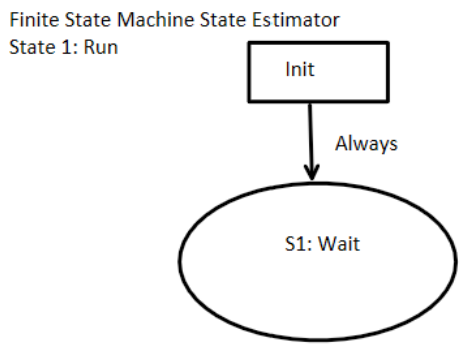

Figure 3. FSM for Sensor Task

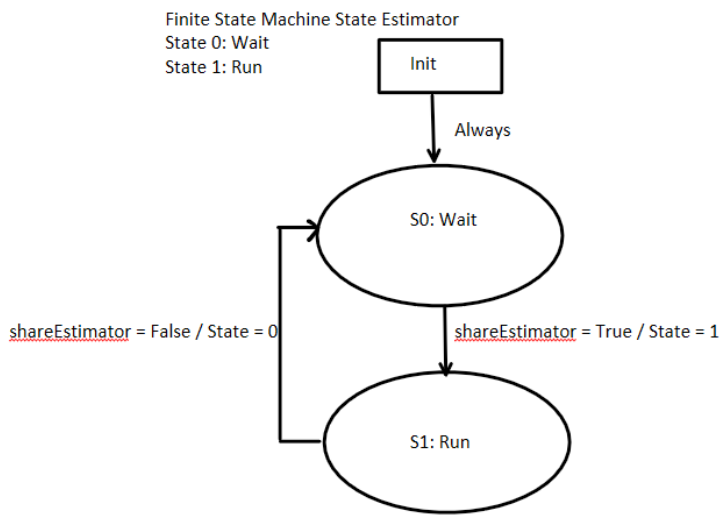

Figure 4. FSM for Estimator Task

Source Code

IMU_class.py:

```
import pyb
from pyb import I2C
from micropython import const
from time import sleep_ms
import os

# --- I2C address options ---
ADDR_A = const(0x28)
ADDR_B = const(0x29)

# --- Registers (subset) ---
```

```
CHIP_ID     = const(0x00)
PAGE_ID     = const(0x07)

ACC_DATA_X_LSB   = const(0x08)
MAG_DATA_X_LSB   = const(0x0E)
GYR_DATA_X_LSB   = const(0x14)
EULER_H_LSB      = const(0x1A)
QUA_W_LSB        = const(0x20)
LIA_DATA_X_LSB   = const(0x28)
GRV_DATA_X_LSB   = const(0x2E)

ACC_OFF_X_LSB = const(0x55)
MAG_OFF_X_LSB = const(0x5B)
GYR_OFF_X_LSB = const(0x61)
ACC_RAD_LSB = const(0x67)

CALIB_STAT  = const(0x35)

OPR_MODE    = const(0x3D)
PWR_MODE    = const(0x3E)
SYS_TRIGGER = const(0x3F)
UNIT_SEL    = const(0x3B)

# --- Operation modes ---
MODE_CONFIG = const(0x00)
MODE_NDOF   = const(0x0C)   # fusion: accel+gyro+mag
MODE_IMU    = const(0x08)

# --- Power modes ---
POWER_NORMAL = const(0x00)

class IMU:
    def __init__(self, i2c, addr=ADDR_A):
        # Using I2C2, device address is 0x28
        self._i2c = i2c
        self._addr = addr
        self._buf = bytearray(8)

    def readbyte(self, reg):
        return self._i2c.mem_read(1, self._addr, reg)[0]
    def readbytes(self, n, reg):
        return self._i2c.mem_read(n, self._addr, reg)
    def writebytes(self, val, reg):
        self._i2c.mem_write(bytes([val & 0xFF]), self._addr,
reg)
    def write2bytes(self, val, reg):
        val &= 0xFFFF
```

```
        lsb = val&0xFF
        msb = (val >> 8) & 0xFF
        self.writebytes(lsb,reg)
        self.writebytes(msb,reg+1)
    @staticmethod
    def combine(lsb, msb):
        v = (msb<<8) | lsb
        return v-65536 if v & 0x8000 else v

    def read_vector(self, startreg, unit_scale):
        data = self.readbytes(6, startreg)
        x = self.combine(data[0], data[1]) / unit_scale
        y = self.combine(data[2], data[3]) / unit_scale
        z = self.combine(data[4], data[5]) / unit_scale
        return x,y,z
    def set_mode(self, mode):
        self.writebytes(mode, OPR_MODE)
        sleep_ms(30)
    def initialize(self, mode=MODE_IMU, reset=True, units =
0x06):
        # units 0x00 gives m/s^2, degrees/sec, Celcius
        self.writebytes(0x00, PAGE_ID)
        sleep_ms(5)
        self.set_mode(MODE_CONFIG)
        # Power Mode: Normal
        self.writebytes(POWER_NORMAL, PWR_MODE)
        sleep_ms(20)
        self.writebytes(units, UNIT_SEL) # select standard units
        sleep_ms(10)
        # Calibration data
        fname = "calibration.txt"
        try:
            os.stat(fname)
            with open(fname, 'r') as file:
                constants = file.read().strip().split(',')
                vals = [int(c) for c in constants if c!=""]
                ACC = vals[0:3]
                MAG = vals[3:6]
                GYR = vals[6:9]
                ACC_RAD = vals[9]
                MAG_RAD = vals[10]
                self.write_coeff(ACC, MAG, GYR, ACC_RAD,
MAG_RAD)
        except OSError:
                self.set_mode(MODE_NDOF)
                while(True):
                    stat = self.calibration_status()
```

```
                    print(stat)
                    if all(bit == 3 for bit in stat[1:]):
                        print("Calibration complete")
                        break
                self.set_mode(MODE_CONFIG)
                ACC_OFF, MAG_OFF, GYR_OFF, ACC_RAD, MAG_RAD =
self.get_coeff()
                lst = list(ACC_OFF) + list(MAG_OFF) +
list(GYR_OFF) + [ACC_RAD, MAG_RAD]
                with open(fname, 'w') as file:
                    file.write(','.join(str(c) for c in lst)+
"\n")
        self.set_mode(mode) # enter IMU mode
    def calibration_status(self, reg=CALIB_STAT):
        # data in the order: system, gyro, accelerometer,
magnetometer
        cbt = self.readbyte(reg)
        system = (cbt >> 6) & 0b11
        gyro = (cbt >> 4) & 0b11
        acc = (cbt >> 2) & 0b11
        mag = cbt & 0b11
        return [system, gyro, acc, mag]
    def get_coeff(self,reg=ACC_OFF_X_LSB):
        # format: [x data, y data, z data]
        raw = self.readbytes(22, reg)
        ACC_OFF = (self.combine(raw[0],raw[1]) &
0xFFFF,self.combine(raw[2],raw[3]) &
0xFFFF,self.combine(raw[4],raw[5]) & 0xFFFF)
        MAG_OFF = (self.combine(raw[6],raw[7]) &
0xFFFF,self.combine(raw[8],raw[9]) &
0xFFFF,self.combine(raw[10],raw[11]) & 0xFFFF)
        GYR_OFF = (self.combine(raw[12],raw[13]) &
0xFFFF,self.combine(raw[14],raw[15]) &
0xFFFF,self.combine(raw[16],raw[17]) & 0xFFFF)
        ACC_RAD = self.combine(raw[18],raw[19]) & 0xFFFF
        MAG_RAD = self.combine(raw[20],raw[21]) & 0xFFFF
        return ACC_OFF,MAG_OFF,GYR_OFF,ACC_RAD,MAG_RAD
    def write_coeff(self, ACC, MAG, GYR, ACC_RAD, MAG_RAD,
reg=ACC_OFF_X_LSB):
        self.write2bytes(ACC[0], reg)
        self.write2bytes(ACC[1], reg+2)
        self.write2bytes(ACC[2], reg+4)
        self.write2bytes(MAG[0], reg+6)
        self.write2bytes(MAG[1], reg+8)
        self.write2bytes(MAG[2], reg+10)
        self.write2bytes(GYR[0], reg+12)
        self.write2bytes(GYR[1], reg+14)
```

```
        self.write2bytes(GYR[2], reg+16)
        self.write2bytes(ACC_RAD, reg+18)
        self.write2bytes(MAG_RAD, reg+20)
    def euler(self, reg=EULER_H_LSB):
        angles = self.read_vector(reg, 900.0)
        return angles
    def ang_v(self, reg=GYR_DATA_X_LSB):
        rates = self.read_vector(reg, 900.0)
        return rates
    def accel(self, reg=ACC_DATA_X_LSB):
        accel = self.read_vector(reg, 100)
        return(accel)
```

task_read_sensor.py:

```
from IMU_class import IMU
from encoder_class import Encoder
from task_share import Share
from micropython import const
from motor_class import Motor

S1_RUN = const(1)

class read_sensors:
    def __init__(self, IMU, EncoderL, EncoderR, MotorL, MotorR,
sL, sR, yaw, yaw_rate, voltageL, voltageR):
        self.IMU : IMU = IMU
        self.EncoderL : Encoder = EncoderL
        self.EncoderR : Encoder = EncoderR
        self.MotorL : Motor = MotorL
        self.MotorR: Motor = MotorR
        self.sL : Share = sL
        self.sR : Share = sR
        self.yaw : Share = yaw
        self.yaw_rate : Share = yaw_rate
        self.voltageL : Share = voltageL
        self.voltageR : Share = voltageR
        self._state = S1_RUN
    def run(self):
        while(True):
            if self._state == S1_RUN:
                self.sL.put(self.EncoderL.get_position())
                self.sR.put(self.EncoderR.get_position())
                self.yaw.put(self.IMU.euler()[0])
                self.yaw_rate.put(self.IMU.ang_v()[2])
                self.voltageL.put(self.MotorL.get_voltage())
                self.voltageR.put(self.MotorR.get_voltage())
            yield self._state
```

task_estimate_state.py:

```
from IMU_class import IMU
from encoder_class import Encoder
from ulab import numpy as np
from micropython import const
from task_share import Share, Queue

S0_WAIT = const(0)
S1_RUN = const(1)

#shareEstimator is a share that sets the state of this task
accordingly

class state_est_task:
    def __init__(self, sL, sR, voltageL, voltageR, yaw,
yaw_rate, shareEstimator, estS, estPsi, estOmegaL, estOmegaR,
dataValsArc, dataValsPsi):
        self.sL : Share = sL
        self.sR : Share = sR
        self.voltageL : Share = voltageL
        self.voltageR : Share = voltageR
        self.yaw : Share = yaw
        self.yaw_rate :Share = yaw_rate
        self.dataValsArc : Queue = dataValsArc
        self.dataValsPsi : Queue = dataValsPsi
        self._state = S0_WAIT
        self._shareEst : Share = shareEstimator
        self._estS : Share = estS
        self._estPsi : Share = estPsi
        self._estOmegaL : Share = estOmegaL
        self._estOmegaR : Share = estOmegaR
        self._encLInitial = 0
        self._encRInitial = 0
        self._headingInitial = 0
        self._xhat = np.zeros((4,1), dtype = np.float) #
initialize state vector
        self._AD = np.array([[0.7427, 0, 0.2494, 0.2494],[0,
0.0061, 0, 0],[-0.1212, 0, 0.3192, 0.3095],[-0.1212, 0, 0.3095,
0.3192]],dtype = np.float)
        self._BD = np.array([[0.1962, 0.1962, 0.1287, 0.1287, 0,
0],[0, 0, -0.0071, 0.0071, 0.0001, 0.0039],[0.7137, 0.4156,
0.0606, 0.0606, 0, -1.8098],[0.4156, 0.7137, 0.0606, 0.0606, 0,
1.8098]],dtype = np.float)
        self._CD = np.array([[1, -70, 0, 0],[1, 70, 0, 0],[0, 1,
0, 0],[0, 0, -0.25, 0.25]], dtype = np.float)
```

```
        self._DD = np.array([[0, 0, 0, 0, 0, 0],[0, 0, 0, 0, 0,
0],[0, 0, 0, 0, 0, 0],[0, 0, 0, 0, 0, 0]], dtype = np.float)
    def run(self):
        while(True):
            if self._state == S0_WAIT:
                if self._shareEst.get() == True:
                    self._state = S1_RUN
            elif self._state == S1_RUN:
                u_star =
np.array([[self.voltageL.get()],[self.voltageR.get()],[self.sL.g
et()],[self.sR.get()],[self.yaw.get()],[self.yaw_rate.get()]],
dtype = np.float)
                xhat_new = np.dot(self._AD,
self._xhat)+np.dot(self._BD, u_star)
                yhat = np.dot(self._CD, self._xhat) +
np.dot(self._DD, u_star)
                self._xhat = xhat_new
                self._estS.put(float(xhat_new[0,0]))
                self._estPsi.put(float(xhat_new[1,0]))
                self._estOmegaL.put(float(xhat_new[2,0]))
                self._estOmegaR.put(float(xhat_new[3,0]))
                if self._shareEst.get() == False:
                    self._state = S0_WAIT
                if not self.dataValsArc.full():
                    self.dataValsArc.put(float(xhat_new[0,0]))
                if not self.dataValsPsi.full():
                    self.dataValsPsi.put(float(xhat_new[1,0]))
            yield self._state
```